Azure Machine Learning

Day 2: Data Preparation & Exploration

Building the foundation for great ML models — from raw data to analysis-ready datasets on Azure.

Day 2 Agenda

What We'll Cover Today

01

Data Preparation Fundamentals

Why it matters and the ML data pipeline

02

Data Handling in Azure ML

Datastores, datasets, versioning, and ingestion

03

Preprocessing Concepts

Missing values, encoding, scaling, and splits

04

Exploratory Data Analysis

Understanding your data before modeling

05

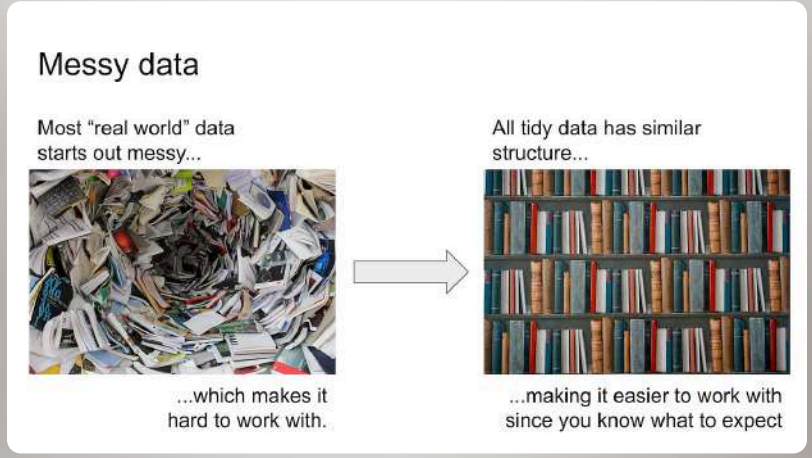
Python Tools

Pandas, Matplotlib, Seaborn, Scikit-learn

06

Hands-on Labs + Cost Tips

Labs 4–6 and Azure cost optimization



CHAPTER 1

Introduction to Data Preparation

Before a model can learn, data must be cleaned, structured, and understood. Data preparation is where most of the real ML work happens.

Why Data Preparation Matters

Garbage In, Garbage Out

Poor data quality directly causes inaccurate predictions — no algorithm can fix bad inputs.

Real-World Data Is Messy

Missing fields, duplicates, inconsistent formats, and outliers are the norm, not the exception.

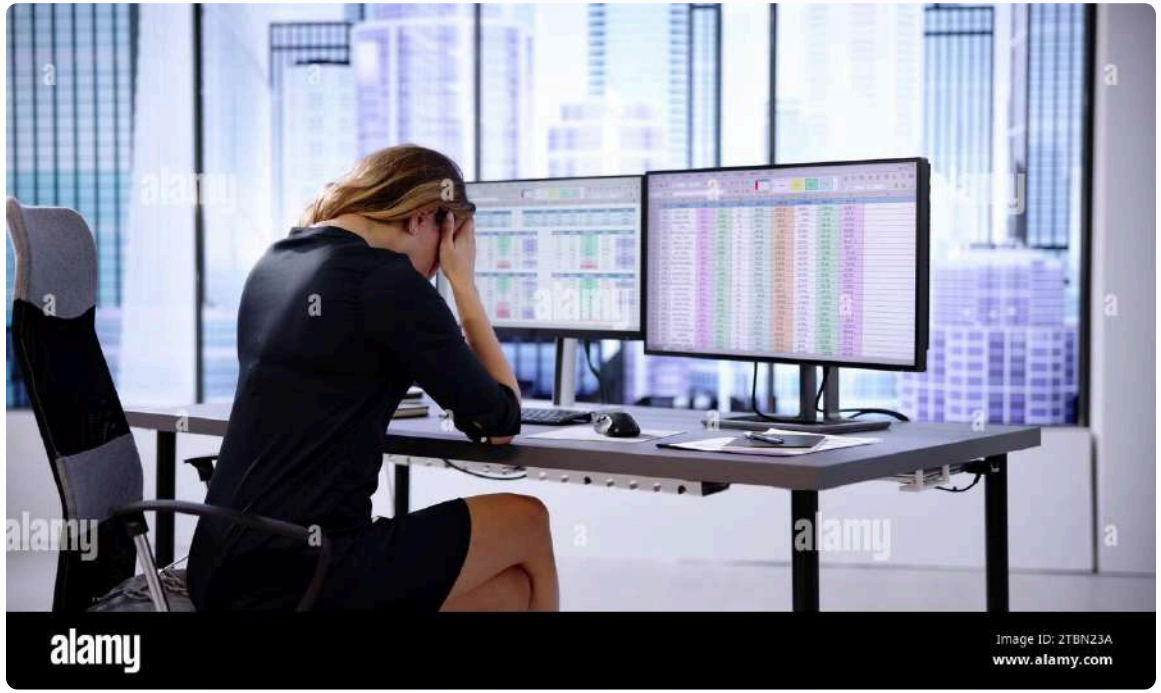
Models Learn from Patterns

Clean, well-structured data enables algorithms to detect meaningful patterns and generalize well.

80/20 Rule in Practice

Data scientists spend ~80% of their time on data preparation — it's the most critical phase of any ML project.

Real-World ML Data Challenges



Missing Values

Sensors fail, forms are skipped, records get lost.

Inconsistent Formats

Dates as "01/01/2024" vs "January 1, 2024" vs timestamps.

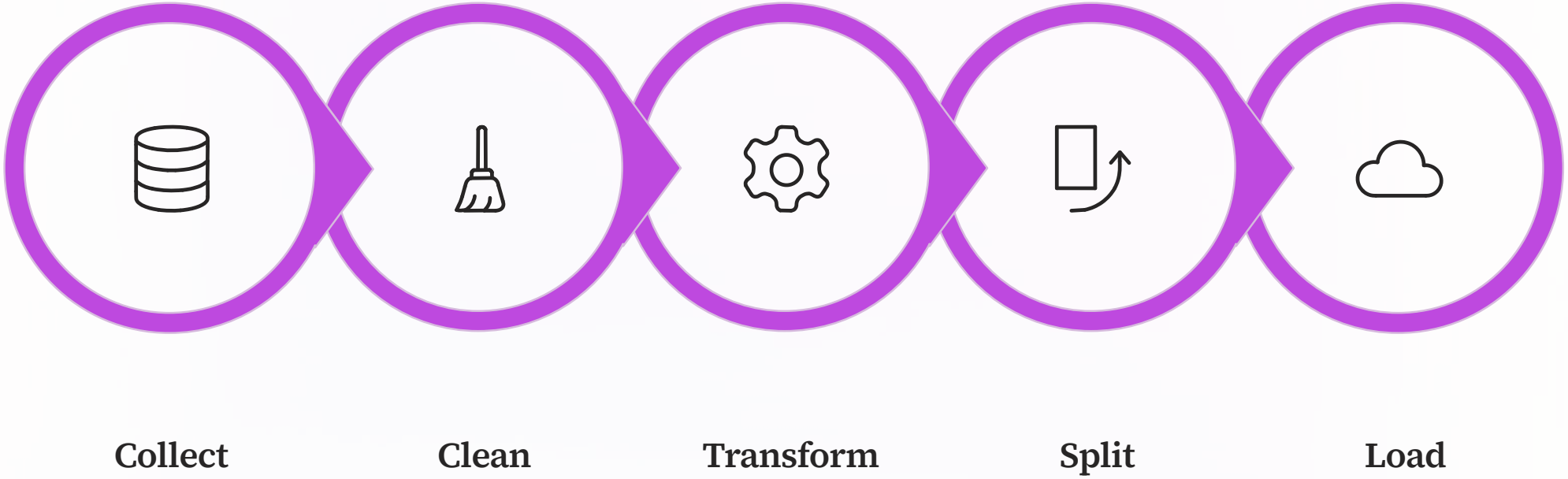
Class Imbalance

Fraud datasets where 99% of records are legitimate — rare events are hard to learn.

Outliers & Noise

Single extreme values can skew model performance significantly.

The ML Data Pipeline Overview

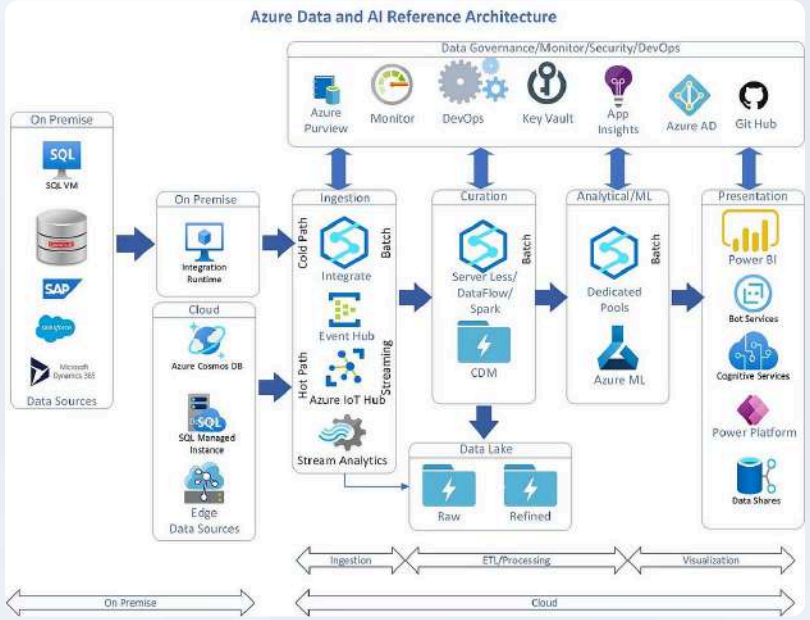


Every ML project follows this pipeline. Skipping or rushing any stage compounds problems downstream — especially during model training and evaluation.

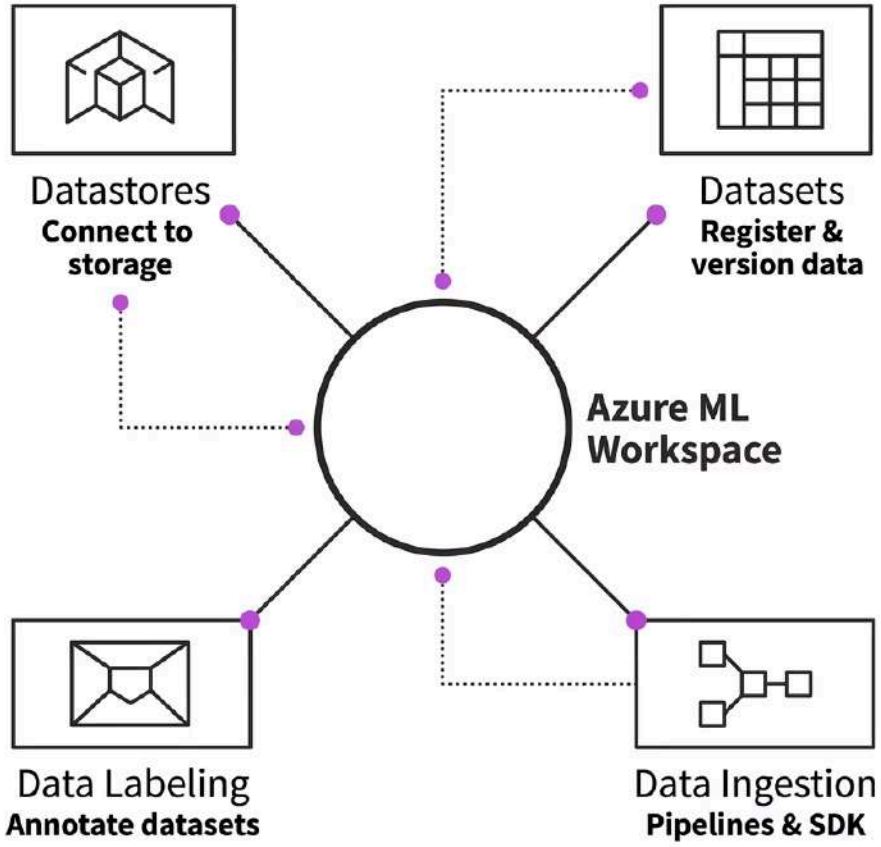
CHAPTER 2

Data Handling in Azure ML

Azure Machine Learning provides a unified workspace for managing data at every stage — from raw ingestion to versioned, model-ready datasets.



Azure ML Data Management Overview



One Workspace, Full Control

Azure ML centralizes all data operations. You connect to storage, register datasets, track versions, and feed pipelines — all from a single workspace without manually moving files.

Azure ML integrates natively with Azure Blob Storage, Data Lake, and SQL databases.

Datastores in Azure ML

What Is a Datastore?

A secure reference to a storage service — no credentials in code. Azure ML handles authentication behind the scenes.

Supported Types

Azure Blob Storage, Azure Data Lake, Azure Files, Azure SQL Database, and more.

Default Datastore

Each workspace comes with a default Blob Storage datastore automatically registered at creation.

Datastores are the bridge between your Azure storage accounts and the Azure ML workspace — keeping sensitive credentials out of notebooks and scripts.

Datasets & Tabular Datasets

What Is an Azure ML Dataset?

A dataset is a **registered, reusable reference** to your data — not a copy. It points to files in your datastore, making it easy to share across experiments and pipelines.

FileDataset

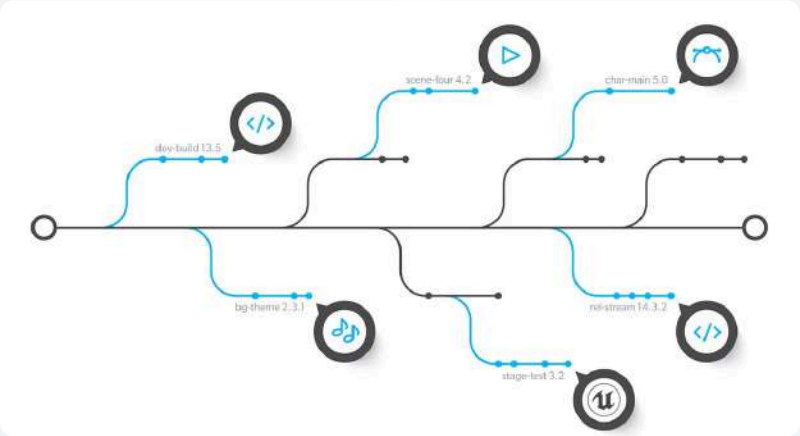
Unstructured files: images, audio, text

TabularDataset

Structured rows: CSV, Parquet, JSON lines

Tabular Datasets

The most common type for structured/CSV data. They load directly into a Pandas DataFrame and support schema inference, column selection, and time-series partitioning.



Data Versioning in Azure ML

1

Register a New Version

Each time data changes, register a new version under the same dataset name.

2

Reproduce Experiments

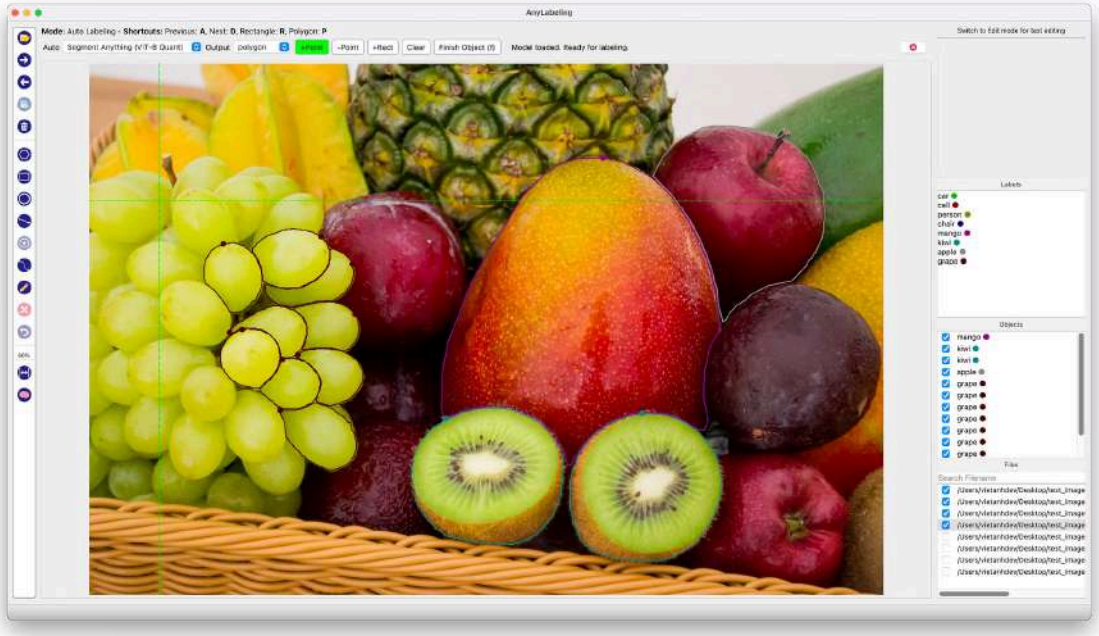
Pin any training run to a specific dataset version for full reproducibility.

3

Audit & Track Changes

View the full history of dataset changes directly in the Azure ML Studio UI.

Data Labeling in Azure ML



What Is Data Labeling?

Supervised ML requires labeled data — each sample needs a correct answer for the model to learn from.

Built-in Labeling Projects

Create image or text labeling projects directly in Azure ML Studio.

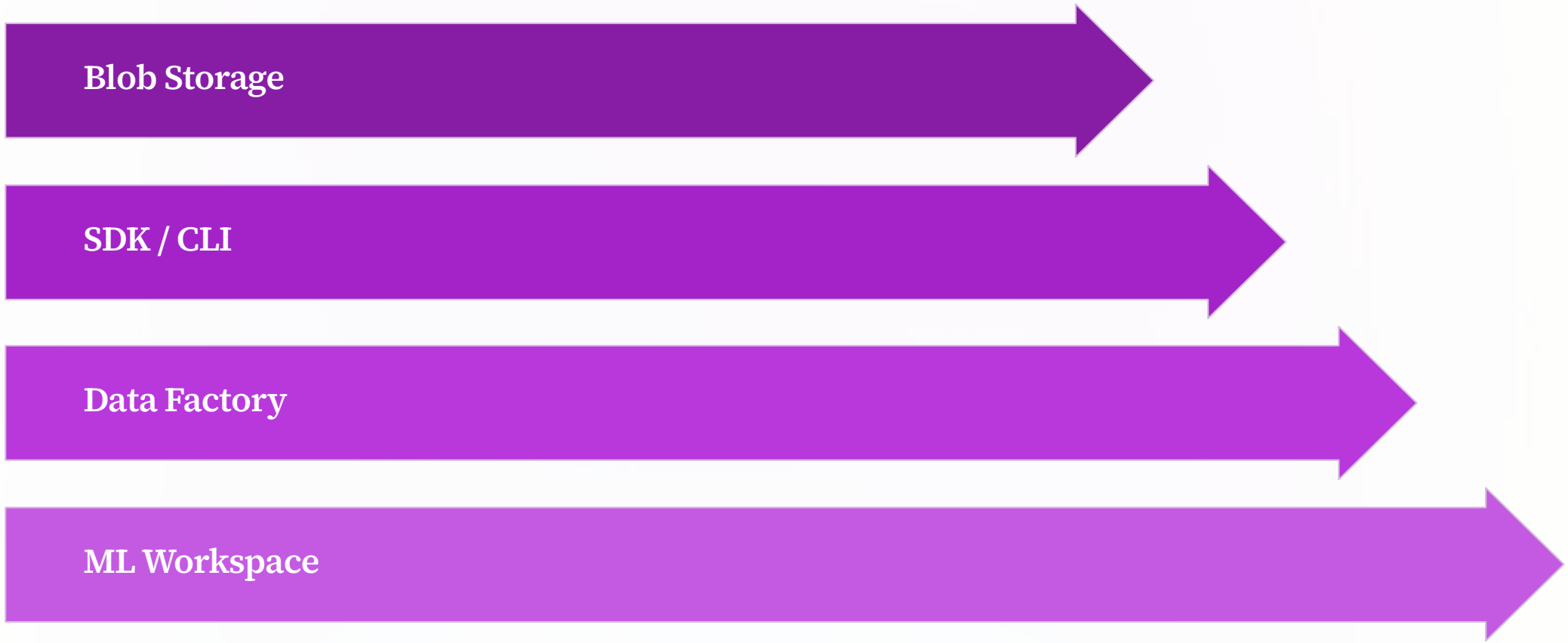
ML-Assisted Labeling

Azure can pre-label samples with a trained model, reducing manual effort significantly.

Export to Dataset

Labeled outputs automatically become registered Azure ML datasets.

Data Ingestion into Azure ML



Data can flow into Azure ML from multiple sources. For Day 2 labs, you'll use Blob Storage and the Python SDK — the most common path for hands-on ML work.

Role of Azure Blob Storage in ML Workflows

Scalable Data Repository

Store raw files, processed CSVs, model artifacts, and experiment outputs — all at low cost and high durability.

Datastore Connection Point

Azure ML datastores reference Blob containers directly — no data movement required for training runs.

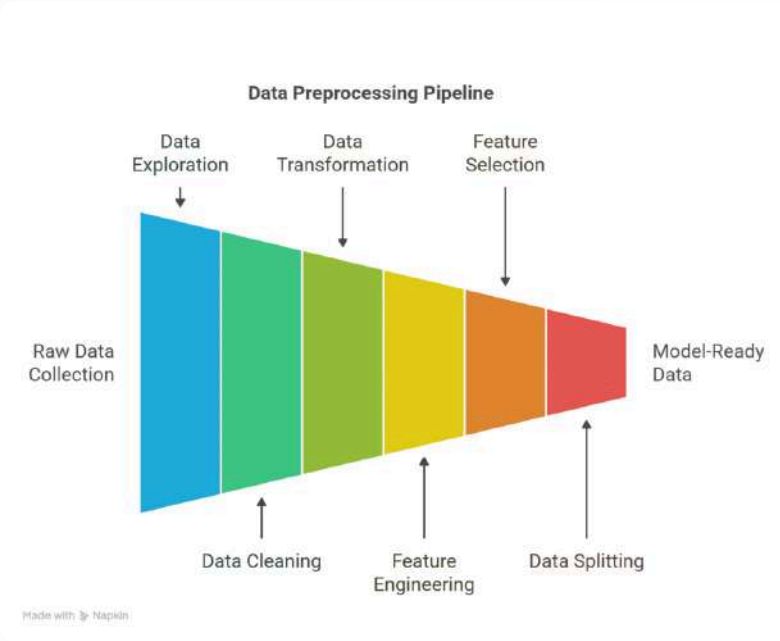
Organize by Project

Use containers and folder paths to separate raw, cleaned, and training-ready data across projects.

CHAPTER 3

Data Preprocessing Concepts

Raw data is rarely model-ready. Preprocessing transforms messy inputs into a consistent, numerical format that ML algorithms can effectively learn from.



Handling Missing Values

Remove Rows

Drop records with missing values when the proportion is very small (<1%)

Mean/Median Fill

Replace missing numbers with column average or median — best for numeric features

Mode Fill

Use the most frequent value for categorical columns

Forward/Back Fill

Propagate adjacent values in time-series data

Python Example

```
import pandas as pd

df = pd.read_csv("data.csv")

# Check for missing values
print(df.isnull().sum())

# Fill numeric columns with mean
df["age"].fillna(df["age"].mean(),
 inplace=True)

# Drop rows missing target
df.dropna(subset=["label"],
 inplace=True)
```

Encoding Categorical Variables

ML algorithms require **numerical inputs**. Categorical text values like "Red", "Blue", or "Male" must be converted to numbers before training.

1

Label Encoding

Maps each category to an integer.

Red→0, Blue→1, Green→2

Best for ordinal categories.

2

One-Hot Encoding

Creates a binary column per category.

Avoids false ordering in nominal data.

Best for unordered categories.

3

Scikit-learn

Use `LabelEncoder` or `OneHotEncoder` from `sklearn.preprocessing` for clean, pipeline-ready encoding.

Feature Scaling

Why Scaling Matters

Features with different ranges (e.g., age: 0–100 vs. salary: 20,000–200,000) can cause algorithms to overweight large-valued features. Scaling puts all features on the same playing field.

Code Example

```
from sklearn.preprocessing import (
    StandardScaler, MinMaxScaler
)

scaler = StandardScaler()

# Fit on training data only
X_train_scaled = scaler.fit_transform(
    X_train
)

# Apply to test data
X_test_scaled = scaler.transform(
    X_test
)
```

Always fit the scaler on training data only — never on test data.

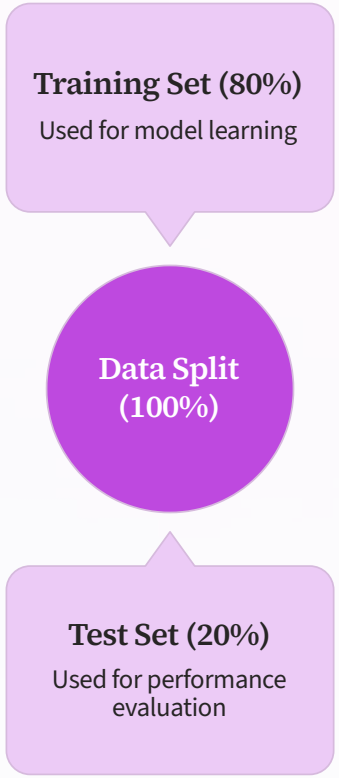
StandardScaler

Mean=0, StdDev=1. Best for normally distributed data.

MinMaxScaler

Scales to [0, 1] range. Best for neural networks.

Train / Test Split



Typical Split

80% training / 20% testing is the most common ratio for general ML tasks.

sklearn Helper

`train_test_split(X, y, test_size=0.2, random_state=42)`

Stratified Split

Use `stratify=y` to preserve class ratios — especially important for imbalanced datasets.

Why Preprocessing Improves ML Models

80%

Time Spent

of an ML project is data work — preprocessing dominates the timeline.

30%+

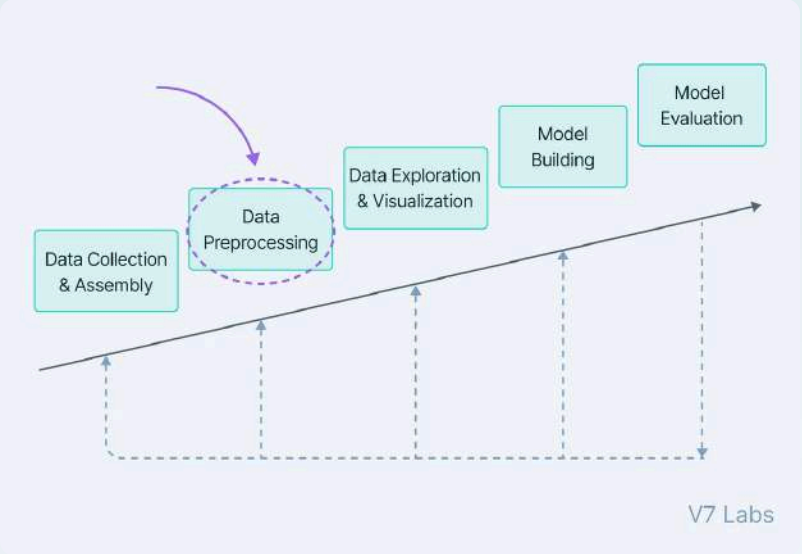
Accuracy Gain

typical improvement in model accuracy when preprocessing is applied properly.

100%

Requirement

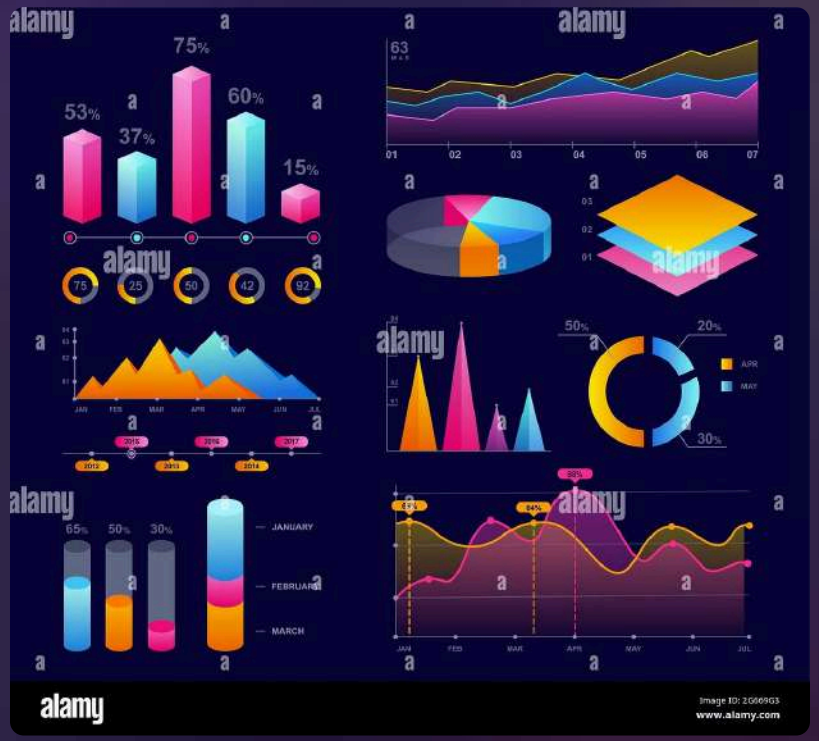
of ML algorithms require numerical, clean, consistently formatted input data.



CHAPTER 4

Exploratory Data Analysis (EDA)

Before building any model, you must **understand your data**. EDA reveals patterns, outliers, and relationships that shape every modeling decision you'll make.



What Is EDA?

Exploratory Data Analysis

EDA is the practice of examining datasets using **statistics and visualizations** to summarize their main characteristics — before making any modeling assumptions.

Pioneered by statistician **John Tukey** in the 1970s, EDA is now a cornerstone of every data science workflow.

> EDA answers: "What does my data look like? What's normal? What's unusual?"

Discover

Find patterns, outliers, and anomalies hidden in raw data.

Validate

Check assumptions about feature distributions and relationships.

Inform

Guide feature engineering, model selection, and data cleaning decisions.

Understanding Dataset Structure

First Steps When Loading Any Dataset

Shape

`df.shape`

→ (rows, columns)

Column Types

`df.dtypes`

→ int, float, object

Summary Stats

`df.describe()`

→ mean, min, max, std

First Rows

`df.head(5)`

→ visual spot-check

Unique Values

`df["col"].value_counts()`

→ category frequency

Checking Missing Values

Code

```
# Total missing values per column
df.isnull().sum()

# Percentage missing
df.isnull().mean() * 100

# Visual heatmap of missing data
import seaborn as sns
import matplotlib.pyplot as plt

sns.heatmap(df.isnull(),
            cbar=False,
            cmap="viridis")
plt.title("Missing Value Map")
plt.show()
```

Interpreting Results

<5% Missing

Usually safe to drop rows or impute without major bias.

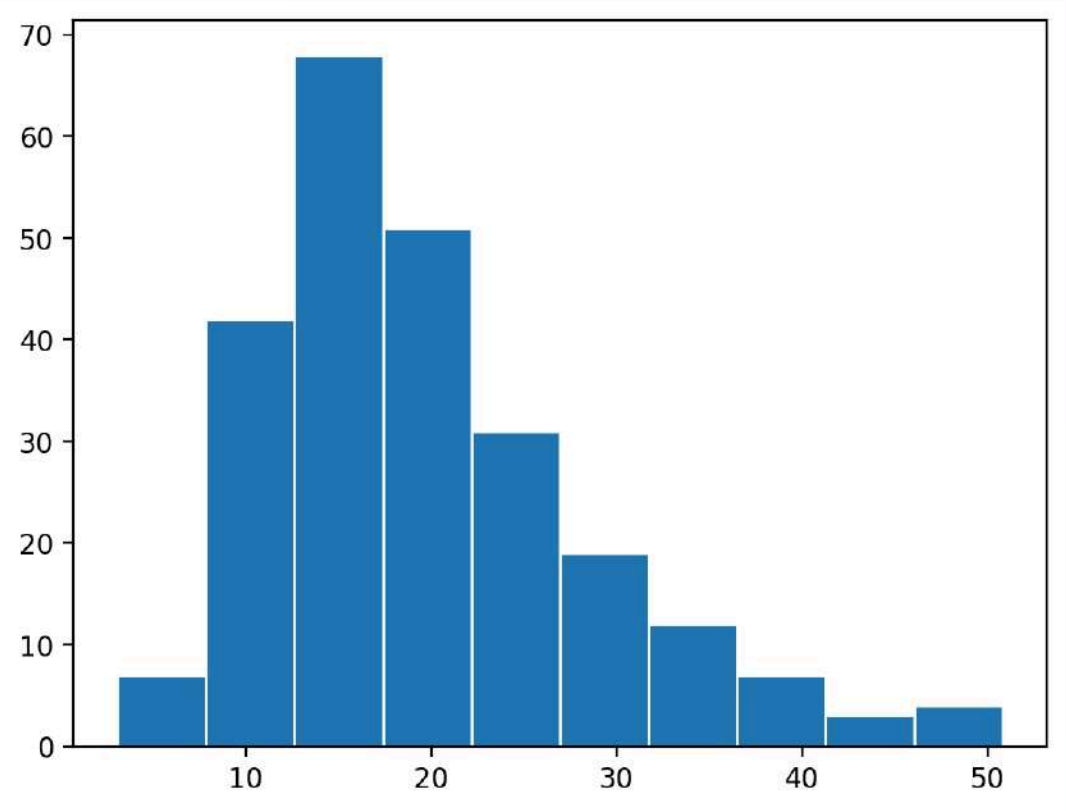
5–30% Missing

Impute carefully — consider model-based imputation.

>50% Missing

Consider dropping the column entirely.

Distribution Analysis



What to Look For

→ **Skewness**

Is the distribution symmetric or skewed? Heavy skew may require log transformation.

→ **Outliers**

Box plots reveal extreme values that may distort model learning.

→ **Normality**

Many algorithms assume normally distributed features — histograms reveal this quickly.

```
df["age"].hist(bins=20)
sns.boxplot(x=df["salary"])
```

Correlation Analysis

What Is Correlation?

Correlation measures how strongly two features move together. Values range from **–1** (perfect negative) to **+1** (perfect positive). Values near **0** indicate no linear relationship.

Code

```
# Compute correlation matrix
corr = df.corr()

# Visualize as heatmap
sns.heatmap(corr,
    annot=True,
    cmap="coolwarm",
    fmt=".2f")
plt.show()
```

High Correlation (>0.8)

Features may be redundant — consider removing one to reduce multicollinearity.

Target Correlation

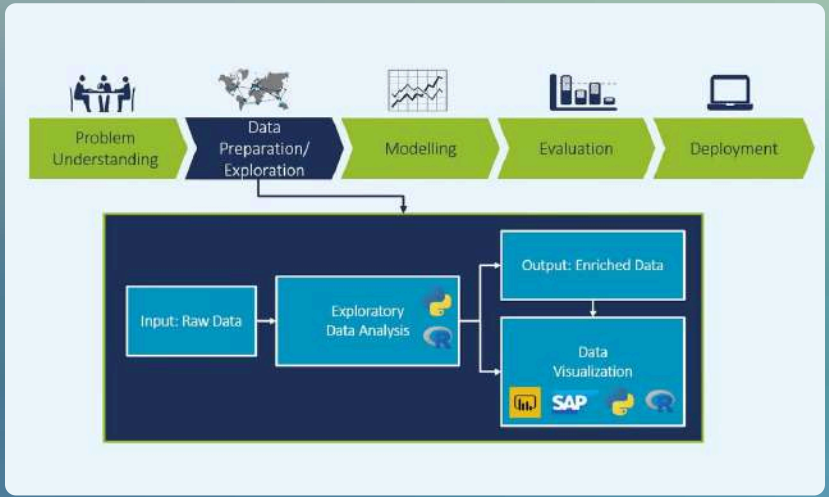
Identify which features have the strongest relationship with the label column.

Negative Correlation

Still a strong signal — as one increases, the other predictably decreases.

Finding Insights from Data

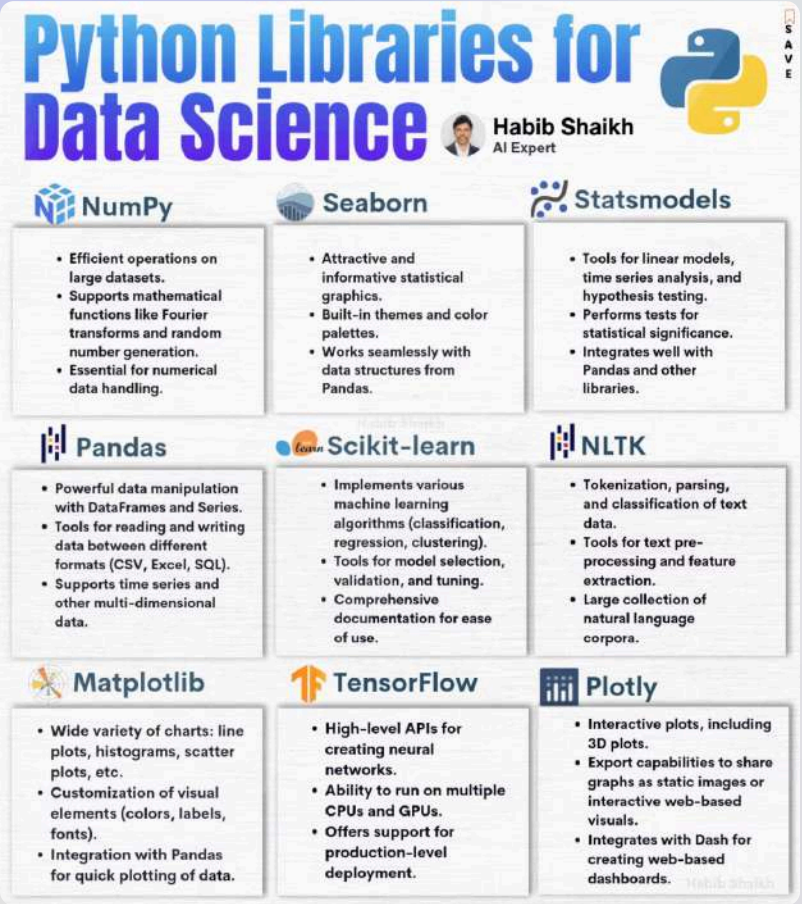
1	**Ask Business Questions** What patterns would make this model useful? What would a wrong prediction cost?
2	**Segment & Group** Use `groupby()` to compare distributions across categories — age groups, regions, product types.
3	**Plot Relationships** Scatter plots and pair plots reveal non-linear interactions that correlation matrices can miss.
4	**Document Findings** Record every anomaly and insight in your notebook — they inform feature engineering in Day 3.



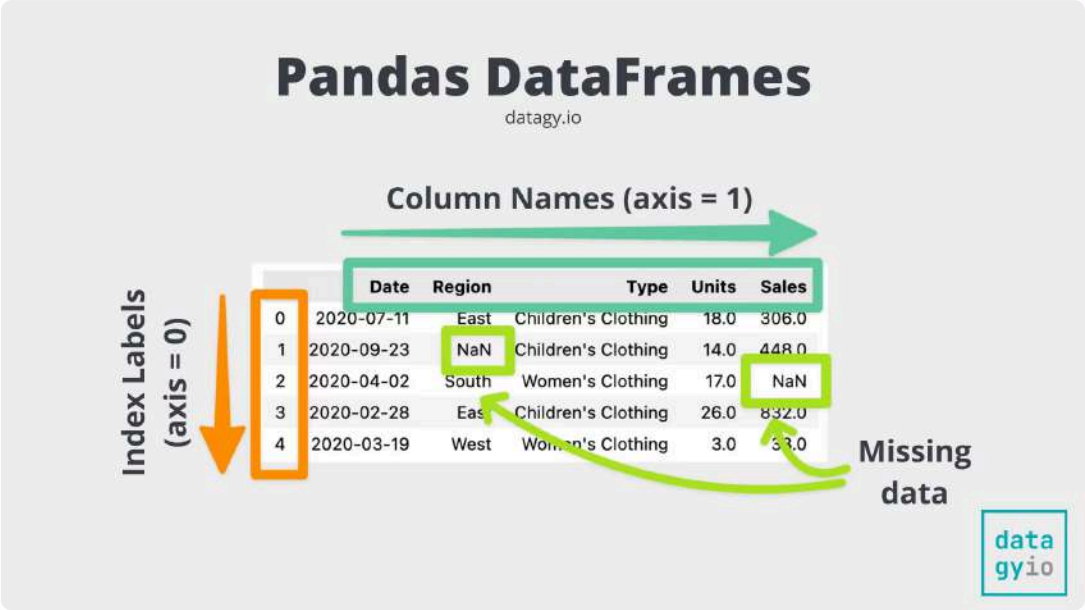
CHAPTER 5

Python Tools for Data Preparation

Four core Python libraries power data preparation and EDA. Together, they cover every step from loading raw CSV files to model-ready feature matrices.



Pandas — Data Manipulation



The Foundation of Data Work

Pandas provides the **DataFrame** — a table-like structure for loading, cleaning, filtering, and transforming data in Python.

→ **Load**

`pd.read_csv()`, `read_excel()`, `read_json()`

→ **Filter**

`df[df["age"] > 30]`

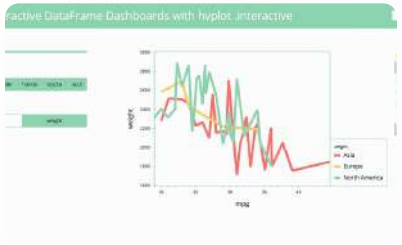
→ **Transform**

`df["income"].apply(np.log)`

→ **Aggregate**

`df.groupby("region").mean()`

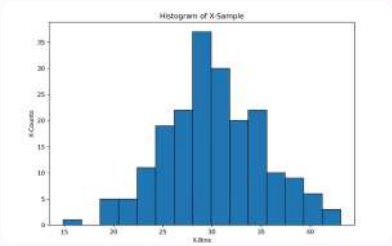
Matplotlib — Data Visualization



Line & Bar Charts

Ideal for trends over time and comparing category values.

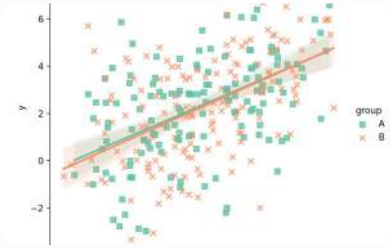
```
plt.plot(x, y)
plt.bar(categories, values)
```



Histograms

Visualize distribution shape, spread, and skewness of numeric features.

```
plt.hist(df["age"], bins=20)
```

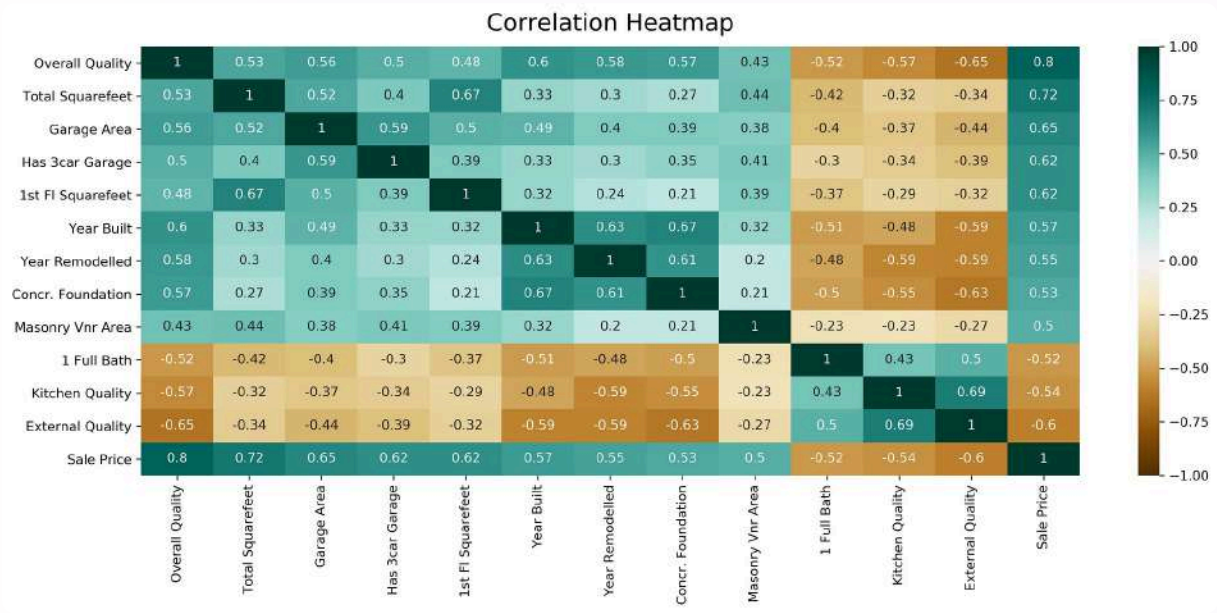


Scatter Plots

Reveal relationships, clusters, and outliers between two numeric features.

```
plt.scatter(df["x"], df["y"])
```

Seaborn — Statistical Plots



Higher-Level Visualizations

Seaborn builds on Matplotlib to provide beautiful, **statistically meaningful** charts with far less code — ideal for EDA.

sns.heatmap()

Correlation matrices with color intensity

sns.boxplot()

Outlier detection and distribution comparison

sns.pairplot()

All pairwise relationships in one command

sns.countplot()

Category frequency — great for class balance check

Scikit-learn — Preprocessing Toolkit

Scikit-learn provides production-ready preprocessing components that plug directly into ML pipelines — no manual transformations needed.

StandardScaler

Normalize feature distributions to zero mean and unit variance for distance-based algorithms.

OneHotEncoder

Convert categorical columns to binary indicator columns for linear models and tree algorithms.

SimpleImputer

Fill missing values using mean, median, or constant strategies in a consistent, repeatable way.

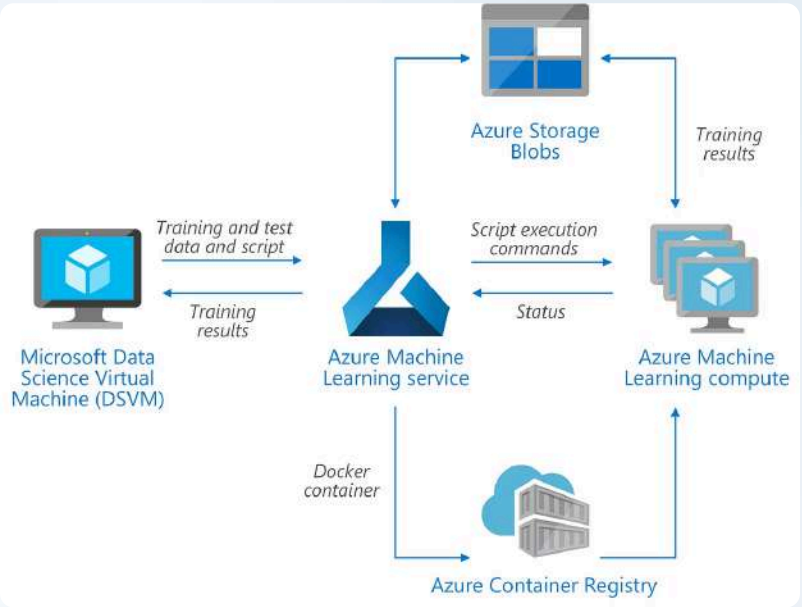
Pipeline

Chain preprocessing steps into a single object — clean, reproducible, and easy to deploy.

CHAPTER 6

Hands-on Labs Overview

Today's three labs give you **hands-on experience** with the full Azure ML data workflow — from uploading raw files to building a preprocessing pipeline in Python.



Lab 4: Upload and Register Dataset

1

Upload File

Upload a sample CSV to Azure Blob Storage via the Azure ML Studio interface.

2

Create Datastore

Register the Blob container as a named datastore in your Azure ML workspace.

3

Register Dataset

Create a TabularDataset pointing to the uploaded file with auto-detected schema.

4

Verify in Studio

Confirm the dataset appears under **Data → Datasets** with correct column types.

Lab Objective

Understand how Azure ML connects to storage and makes data accessible across experiments without duplicating files.

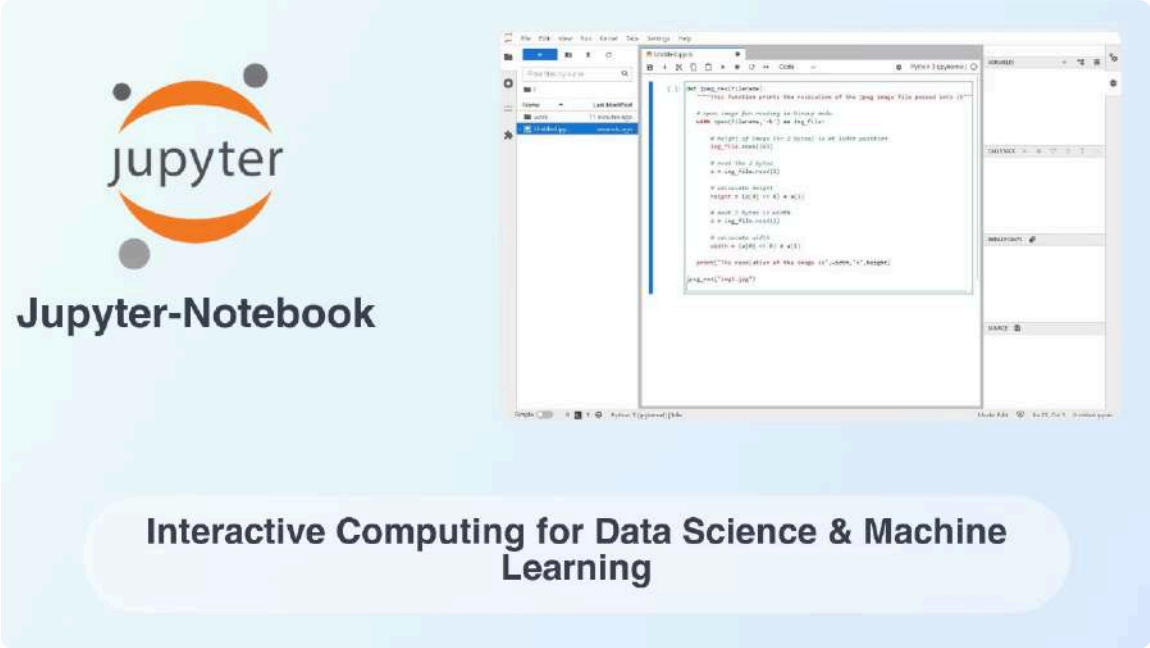
Expected Outcome

A registered TabularDataset visible in Azure ML Studio, ready to be used in notebooks and training scripts.

Dataset Used

Titanic CSV or a provided sample dataset with mixed numeric and categorical columns.

Lab 5: Data Exploration Using a Notebook



Objective

Perform a full EDA on a registered Azure ML dataset using Pandas, Matplotlib, and Seaborn inside an Azure ML Compute Notebook.

Key Tasks

Expected Output: A notebook with EDA cells, annotated charts, and written observations.

Load Dataset

Read the registered dataset into a Pandas DataFrame using the SDK.

Profile the Data

Run `df.describe()`, `df.isnull().sum()`, and `df.dtypes`.

Visualize

Plot histograms, box plots, and a correlation heatmap.

Lab 5: Load Dataset via SDK

Getting Your Registered Dataset into a Notebook

```
from azureml.core import Workspace, Dataset

# Connect to your workspace
ws = Workspace.from_config()

# Load the registered dataset by name
dataset = Dataset.get_by_name(
    ws, name="titanic-dataset"
)

# Convert to Pandas DataFrame
df = dataset.to_pandas_dataframe()

# Quick profile
print(df.shape)
print(df.describe())
print(df.isnull().sum())
```

Run this in an Azure ML Compute Notebook — your workspace config is auto-available in that environment.

Lab 6: Data Preprocessing Pipeline

Objective

Build a reusable, scikit-learn preprocessing pipeline that handles missing values, encodes categories, and scales features — then export the cleaned dataset.

Tasks

01

Identify Columns

Separate numeric and categorical feature columns.

02

Build Transformers

Use `SimpleImputer`, `StandardScaler`, and `OneHotEncoder`.

03

Create Pipeline

Use `ColumnTransformer` + `Pipeline` to chain all steps.

04

Apply & Save

Fit on training data, transform both splits, and save the output CSV.

Expected Outcome

A clean, processed dataset saved to your datastore — ready for Day 3 model training.

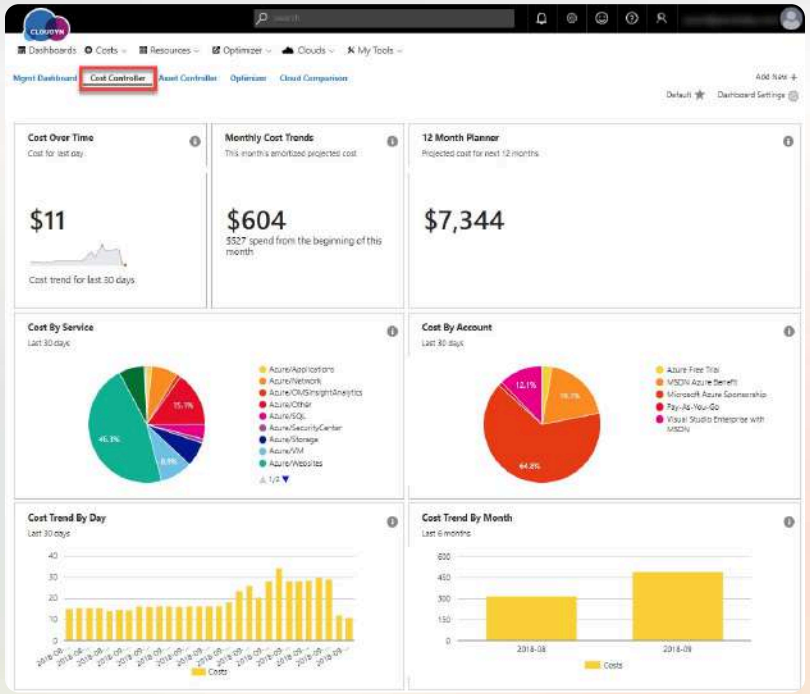
Pipeline Code Snippet

```
from sklearn.pipeline import Pipeline
from sklearn.compose import (
    ColumnTransformer
)
from sklearn.preprocessing import (
    StandardScaler, OneHotEncoder
)
from sklearn.impute import SimpleImputer

num_pipe = Pipeline([
    ("impute", SimpleImputer(
        strategy="mean")),
    ("scale", StandardScaler())
])

cat_pipe = Pipeline([
    ("impute", SimpleImputer(
        strategy="most_frequent")),
    ("encode", OneHotEncoder(
        handle_unknown="ignore"))
])
```

Lab 6: Combining into ColumnTransformer

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

# Define column groups
numeric_cols = ["age", "fare", "pclass"]
categorical_cols = ["sex", "embarked"]

# Combine transformers
preprocessor = ColumnTransformer([
    ("num", num_pipe, numeric_cols),
    ("cat", cat_pipe, categorical_cols)
])

# Full preprocessing pipeline
full_pipeline = Pipeline([
    ("preprocess", preprocessor)
])

# Fit and transform
X_train_ready = full_pipeline.fit_transform(
    X_train
)
X_test_ready = full_pipeline.transform(
    X_test
)

print("Training shape:", X_train_ready.shape)
print("Test shape:", X_test_ready.shape)
```

This pipeline will be reused on Day 3 for model training — keep it saved in your notebook.



CHAPTER 7

Cost Optimization Tips in Azure ML

Azure charges for compute time, storage, and data transfer. Following a few simple habits keeps lab costs near zero while you learn.

Use Small Datasets for Development

Why It Matters

Training on a 10,000-row sample instead of 1 million rows can be **100× faster** and cost **a fraction** of the full run — while still validating that your code works correctly.

Practical Approach

Sample During Development

Use `df.sample(n=1000)` during EDA and pipeline testing.

Full Data for Final Runs

Only switch to the full dataset when your pipeline is verified and working.

CPU vs. GPU Compute

✅ Use CPU for Labs

Data preprocessing, EDA, and classic ML (scikit-learn, XGBoost) run perfectly on standard CPU clusters.

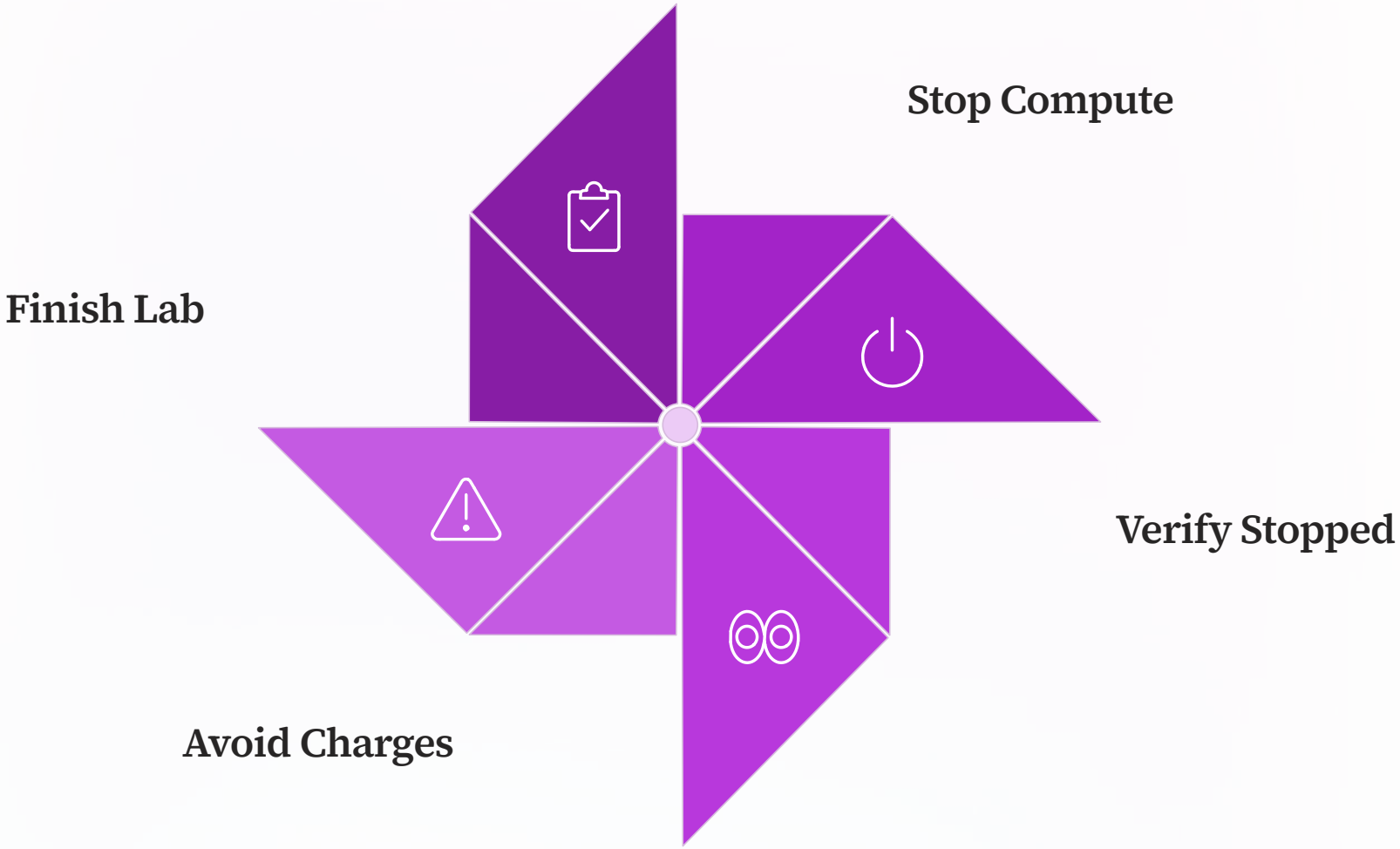
Recommended: **Standard_DS2_v2** or **Standard_DS3_v2**

⚠️ GPU Is for Deep Learning

Reserve GPU instances (NC-series) for computer vision and NLP training only — they cost 5–10× more per hour.

Using GPU for pandas or sklearn is wasteful spending.

Stop Compute After Every Lab



Reminder: A compute instance left running overnight can accumulate $10–$30+ in unexpected charges. Always stop it when done — even during lunch breaks.

More Cost Optimization Habits

Clean Up Unused Storage

Delete intermediate CSV files, test uploads, and unused dataset versions from your Blob containers regularly.

Set Budget Alerts

Use Azure Cost Management to set a spending alert — get notified via email before your budget is exceeded.

Delete Failed Runs

Completed and failed experiment runs store logs and outputs — clean them up periodically to save storage costs.

Use Low-Priority Compute

For non-time-sensitive jobs, low-priority clusters can reduce compute costs by up to 80% compared to dedicated instances.

CHAPTER 8

End of Day 2 Summary

You've covered the complete data journey — from raw, messy inputs to clean, structured, model-ready datasets using Azure ML and Python.

Key Concepts Learned Today

Data Preparation

Why it matters, real-world challenges, and the ML data pipeline from collection to model input.

Azure ML Data Tools

Datastores, TabularDatasets, versioning, labeling, and data ingestion workflows.

Preprocessing

Missing value handling, encoding, scaling, and train/test splitting using scikit-learn.

EDA

Profiling data structure, distributions, correlations, and finding actionable insights.

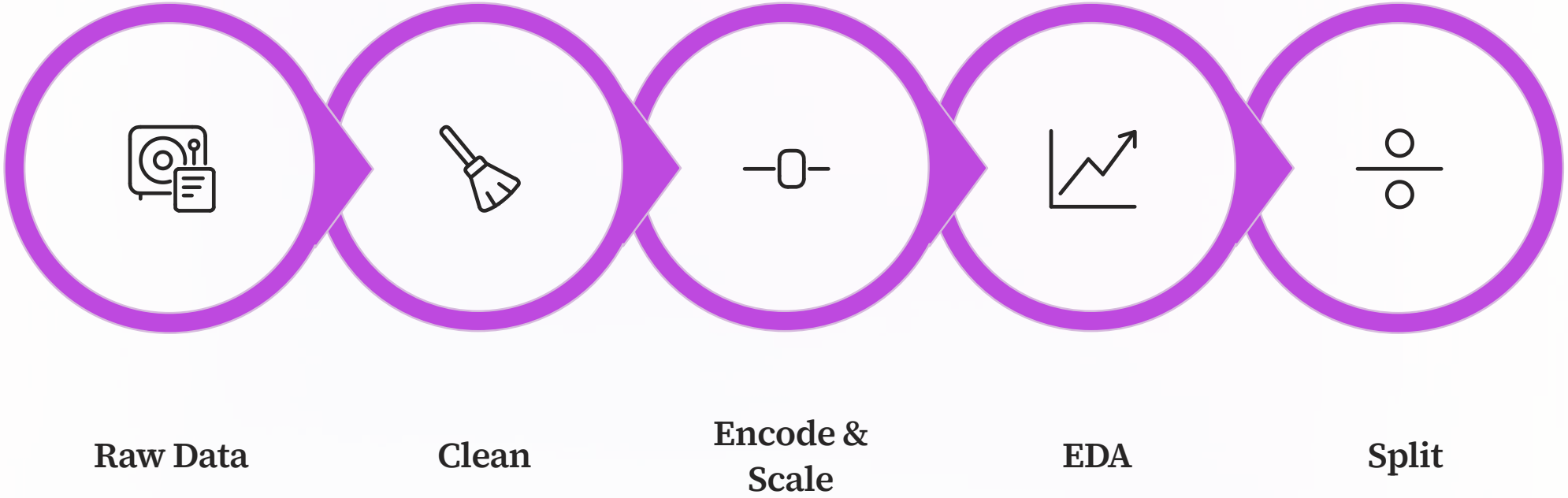
Python Tools

Pandas, Matplotlib, Seaborn, and Scikit-learn for end-to-end data preparation.

Cost Habits

Small datasets, CPU compute, stopping instances, and budget alerts to minimize Azure spend.

Data Preparation Workflow for ML



This workflow is your **repeatable playbook** for every ML project. Internalize it now — Day 3 will pick up exactly where this diagram ends: the preprocessed dataset going into model training.

Lab Recap: What You Built Today

1

Lab 4 — Dataset Registration

Uploaded a CSV to Azure Blob Storage, created a datastore connection, and registered a TabularDataset in Azure ML Studio.

2

Lab 5 — Data Exploration

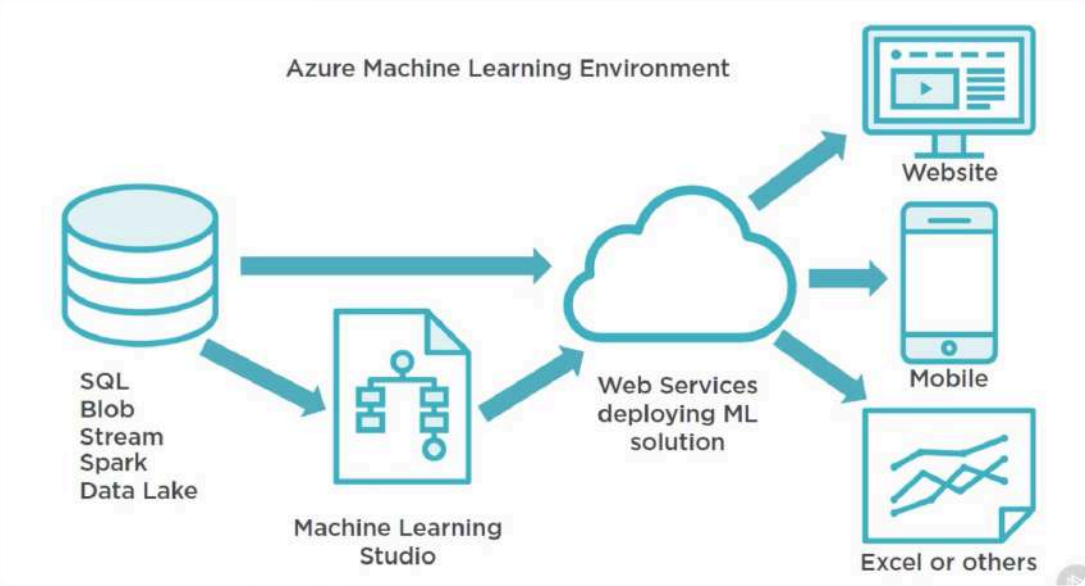
Loaded the registered dataset into a compute notebook, profiled its structure, and visualized distributions and correlations using Seaborn and Matplotlib.

3

Lab 6 — Preprocessing Pipeline

Built a scikit-learn `Pipeline` with imputation, encoding, and scaling — producing a clean, model-ready dataset saved to your datastore.

Coming Up: Day 3 — Model Training



What's Next

With clean, preprocessed data in your datastore, you're ready to train your first Azure ML model. Day 3 covers:

→ **ML Algorithms**

Classification, regression, and when to use each.

→ **Training in Azure ML**

Submitting experiments, tracking metrics, and logging runs.

→ **AutoML**

Let Azure automatically select and tune the best model for your dataset.

→ **Model Evaluation**

Accuracy, precision, recall, confusion matrix, and AUC-ROC interpretation.

Questions & Discussion

> "Good data preparation is not a phase you complete — it's a discipline you practice on every project."

Take a moment to review your Lab 6 preprocessing pipeline. Make sure it runs end-to-end and your cleaned dataset is saved. You'll need it first thing on Day 3.

See You on Day 3!

📁 Before You Go

Stop your compute instance, save your notebooks, and confirm your preprocessed dataset is registered in Azure ML Studio.

📚 Optional Reading

Azure ML Data documentation, Pandas user guide, and Scikit-learn preprocessing reference — links shared in course materials.

🚀 Day 3 Preview

Model Training in Azure ML — bring your preprocessed dataset and your curiosity. We build and evaluate models next!